

## Assignment on Predictive Analytics

# Stock Price Prediction | Sales Forecasting | Waiter Tips Regression

Name: Rabea Akter Shoily Student ID: 905253001

### GitHub repository:

<https://github.com/Shoily01/Stock-Price-Prediction-Sales-Forecasting-and-Waiter-Tips-Regression.git>

## 1. Introduction

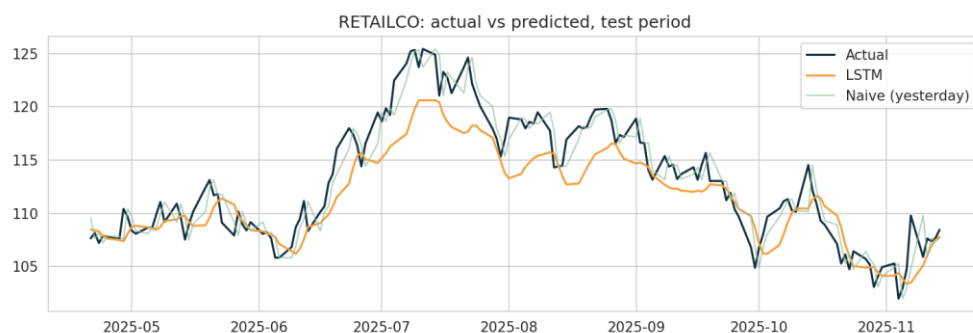
This report is a summary of three predictive models created using the synthetic data set: one LSTM neural network model for the stock price, time-series models for the store sales, and regression models for the waiter tips. The same process was repeated over and over: Explore the data, prepare the data, train, test on data not used during training, always against a simple benchmark. The code, saved pipeline, and charts are in the GitHub repository linked above.

## 2. Models and results

### 2.1 Stock price prediction (LSTM)

**Table 1. Stock model results on the held-out test period (RETAILCO)**

| Model                  | RMSE  |
|------------------------|-------|
| LSTM (30-day window)   | 2.744 |
| Naive (previous close) | 1.624 |



*Figure 1. Actual vs predicted RETAILCO price, test period*

The strategy involved trying to forecast the closing price of the day using a two-layer LSTM network with a window of 30 days of past closing prices, dropout rate of 0.2 and early stopping. The scaler for the price was fitted only on training rows and the first 80 days of the data were used for training and the remaining 150 days for testing. The performance of the LSTM was not superior to the simple method “tomorrow = today” (RMSE 2.74 vs. 1.62). Test prices were 125.4, while a network trained with scaled prices performs poorly outside the range of the training prices, with a maximum price of 117.4. This is a prediction, not an error in coding: predicting prices can be challenging, and “yesterday's price” is a difficult indicator.

## 2.2 Sales forecasting

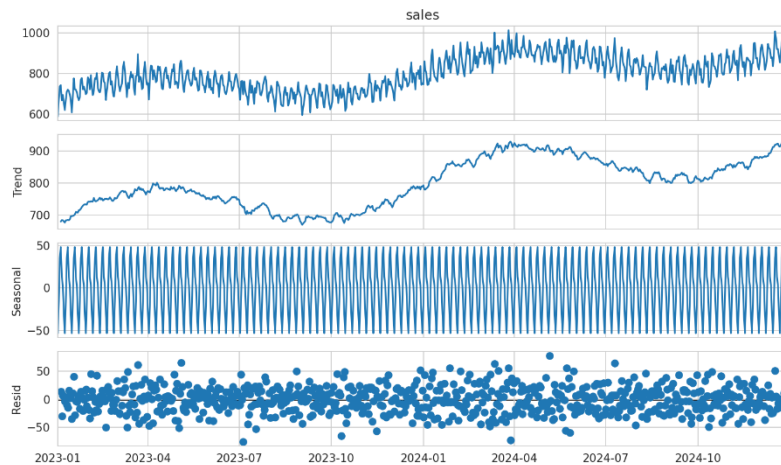


Figure 2. Seasonal decomposition of East store sales

Table 2. Sales forecast results on the last 60 days (East store)

| Model                  | RMSE (units/day) |
|------------------------|------------------|
| Seasonal naive         | 60.31            |
| Holt-Winters           | 24.07            |
| SARIMA(1,1,1)(1,0,1,7) | 44.27            |

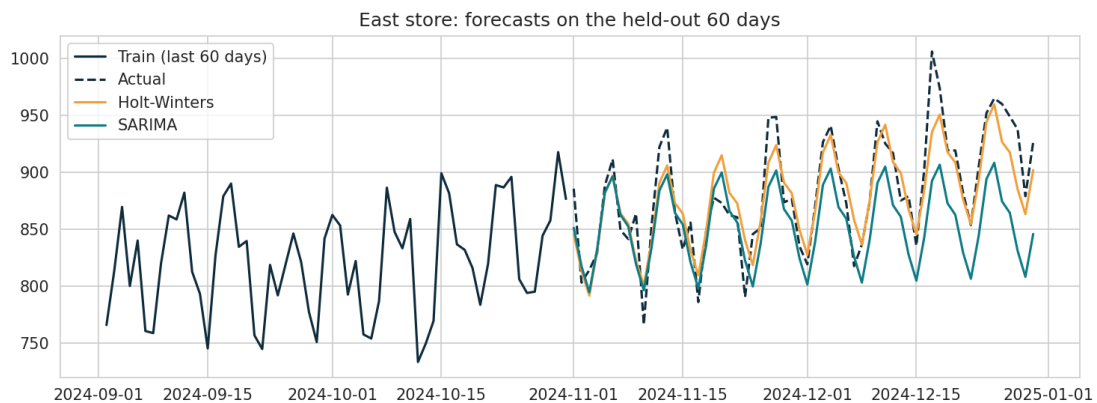


Figure 3. Forecasts against actual sales, held-out 60 days

**Trading the large timeframes of trend and weekly chart patterns:** Figure 2 illustrates that there is a steady upward trend (around 680 to 939 units during the 2 year period) and a weekly time cycle that repeats every 7 days. High midweek (Wednesday +48, Tuesday +35, Thursday +13 against average) and low weekend and Monday (Sunday 54, Saturday 31, Monday 16).

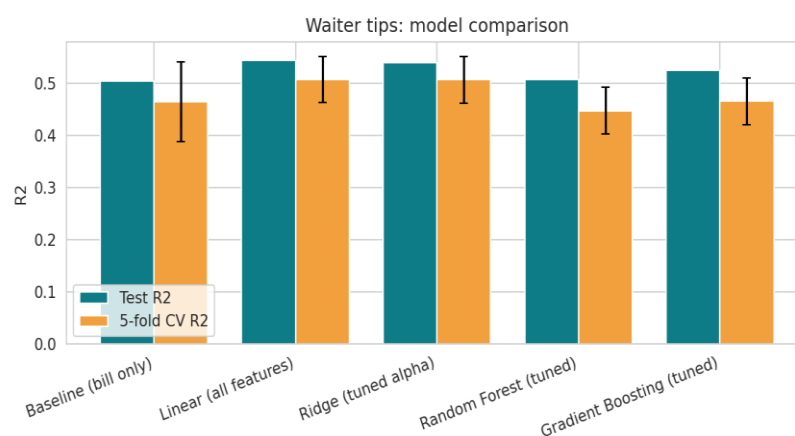
**Best model:** Holt-Winters beat SARIMA and seasonal naive with RMSE of 24.07 vs 44.27 and 60.31 respectively. The only thing that is simple about the series is the smooth trend, the regular weekly pattern and random noise, the same elements that Holt-Winters models with a very few parameters. Seasonal naive repeats the previous week, meaning it has no knowledge of trend, just noise. There is also a level of noise: when the noise component is removed from the data (trend and season), the

remaining noise has a standard deviation of approximately 23, and the errors made by Holt-Winters are nearly the same size (23.9). So, it is already almost at the optimum performance of a model.

## 2.3 Waiter tips regression

**Table 3. Tips model comparison, baseline to most refined**

| Model                        | Train $R^2$ | Test $R^2$ | Test RMSE | CV $R^2$ (mean) |
|------------------------------|-------------|------------|-----------|-----------------|
| 1. Baseline (bill only)      | 0.463       | 0.503      | 1.024     | 0.464           |
| 2. Linear (all features)     | 0.534       | 0.544      | 0.981     | 0.506           |
| 3. Ridge (tuned alpha)       | 0.523       | 0.539      | 0.986     | 0.506           |
| 4. Random Forest (tuned)     | 0.594       | 0.507      | 1.020     | 0.447           |
| 5. Gradient Boosting (tuned) | 0.597       | 0.524      | 1.003     | 0.465           |



*Figure 4. Test and cross-validated  $R^2$  by model*

**Improvement from baseline:** Test  $R^2$  rose from 0.503 to at most 0.544 (+0.04), and cross-validated  $R^2$  from 0.464 to 0.506. Some of the variation in tips can be accounted for by bill size alone (correlation 0.69), with the other columns contributing little. The remainder represents personal behavior which isn't recorded in the data. Random Forest, with an average score of ~0.59 on training data, generally did not fare better on test data (0.507, 0.524), nor in cross-validation (0.447, 0.465) suggesting some degree of overfitting on only 300 rows. The number of rows in the test set is very low (60) in this case, and cross validation is the more consistent indicator.

**What drives tips:** The largest effect is total bill (+0.83), followed by party size (+0.40: for every additional person in the party, bill increases by approximately \$0.40) and smoker status (approximately \$0.80: smokers pay \$0.80 less). The other effects are close to zero with Sunday contributing a small positive (+0.27). The total\_bill, the smoker and the size (0.01) and the day (0.004) do not seem to be significant. total\_bill is far more important than the smoker or the size (0.01) or the day (0.004) or the sex or the time (0.00). To sum it up, the tips are generally a percentage of the bill.

**The model created to deploy: Ridge Regression.** Ridge has the lowest error rate (train-test: 0.523 vs 0.539), and the lowest  $R^2$  (0.506) when trained on cross-validated data; the regularisation is able to handle the related inputs (total bill and bill per person); it has not overfitted. It's also simple and easy to get across to the restaurant manager. The tree models are more complicated, but no better. The one of

these models that performed the best (by more than 0.02 CV  $R^2$ ) was Ridge (which is what the rule selected).

### 3. The split should be based on time, not distance

The data for stocks and sales is recorded in (chronological) order of time, and the value of today's data is closely related to yesterday's. Suppose a random split gave the 10<sup>th</sup> June a training window, and the 11<sup>th</sup> June a window of testing, then the model has seen the answer before it even gets started. The score would be great but would have no meaning, since in future use tomorrow's data will never exist. Copy to the past- predict the future: a time-based split. The scaler was also only installed on training rows to ensure there is no leakage from the test period. The tips are suitable for random splitting since there is no time order of events.

### 4. Limitations and improvements

Synthetic data, a single ticker, a single store. The results may not be applicable to an actual market or other retailers. All Tickers and stores should be repeated on real data to judge for improvement.

- **The price is forecasted by LSTM and goes beyond its training window.**  
Improvement: Predict daily returns percentage changes instead add volume as a feature and compare once again.
- **One 60-day window was used for sales testing, and SARIMA was not tuned.**  
Improvement: Implement rolling-origin validation and SARIMA orders for search automatically, or introduce variables of holiday and promotion.
- **Only 300 tip rows, and tuned-tree CV scores are somewhat optimistic** due to the fact that they were optimized with the same training data. This should be improved by collecting more bills and using nested cross-validation.

### 5. Deployment

**Tips pipeline:** To ensure that everyone that loads the feature engineering, encoding, scaling, and the Ridge model get the same preprocessing, it was saved as a single file (tips\_pipeline.joblib). When it was reloaded, it gave the same prediction as the original. On three unseen bills it predicted \$6.81 for a \$45 Saturday dinner for four (15.1%), \$2.71 for an \$18.50 Thursday lunch for a smoking party of two (14.6%) and \$5.34 for a \$32 Sunday dinner for three (16.7%). These are reasonable about 15% of the bill, the smoking party is slightly lower.

**LSTM and Holt-Winters:** A model file (lstm\_retailco.keras) and scaler (lstm\_scaler.joblib) are already saved, and these would be loaded together. It's not time-consuming and can be saved or just refitted.

Predicting tips based on API requests. A small service loads the pipeline once and returns back a tip for every bill sent to it.

**Scheduled (daily batch):** Load daily last 30 closes, scale, run the LSTM and store the prediction for the next day after market close. The sales job refits Holt-Winters every night or week based on the latest

information and prepares a 60-day table of forecasts. Retrain the LSTM every month, and monitor live RMSE relative to the naive model.

## 6. Conclusion

**Stock:** if the price data fell outside of the range used to train the LSTM, then the returns should be modelled next. **Sales:** Holt-Winters identified and exploited the trend and weekly cycle and got to the noise floor (RMSE 24.07). Refinements were added to the tip, but were quite small and had only about 0.04  $R^2$ , so the simple and explainable Ridge model was used.

---

**The End**